CODE MENTOR GPT: A HYBRID AI FRAMEWORK FOR EDUCATIONAL CODE REVIEW AND INTELLIGENT COMMENT GENERATION

Project Phase-I Report submitted to Osmania University, Hyderabad, in partial fulfillment of the requirement for the award of the degree of

Bachelor of Engineering

in

Information Technology

By

Mr. Mohammed Omer Farooq (160923737048)

Mr. Mohammed Zeshan Mohiuddin (160923737053)

Under the Guidance of

Mr. Yellaiah Ponnam

Assistant Professor

DEPARTMENT OF INFORMATION TECHNOLOGY

LORDS INSTITUTE OF ENGINEERING AND TECHNOLOGY

(UGC Autonomous)

(Affiliated to Osmania University)

Himayathsagar, Hyderabad, 500091

A.Y. 2026-2027



LORDS INSTITUTE OF ENGINEERING AND TECHNOLOGY

(UGC Autonomous)

(Affiliated to Osmania University)

Himayathsagar, Hyderabad, 500091

Department of Information Technology

CERTIFICATE

This is to certify that the Project Phase-I report entitled **CODE MENTOR GPT: A HYBRID AI FRAMEWORK FOR EDUCATIONAL CODE REVIEW AND INTELLIGENT COMMENT GENERATION** being submitted by **Mr. Mohammed Omer Farooq (160923737048)** and **Mr. Mohammed Zeshan Mohiuddin (160923737053)** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Engineering** in **Information Technology** is a record of bonafide work carried out by them.

Internal Project Guide — **Dr. K. Nagi Reddy**

Mr. Yellaiah Ponnam — **H.O.D, I.T**

Assistant Professor

External Examiner



LORDS INSTITUTE OF ENGINEERING AND TECHNOLOGY

(UGC Autonomous)

(Affiliated to Osmania University)

Himayathsagar, Hyderabad, 500091

Department of Information Technology

DECLARATION BY THE CANDIDATES

We, **Mr. Mohammed Omer Farooq (160923737048)** and **Mr. Mohammed Zeshan Mohiuddin (160923737053)**, hereby declare that the Project Phase-I report entitled **CODE MENTOR GPT: A HYBRID AI FRAMEWORK FOR EDUCATIONAL CODE REVIEW AND INTELLIGENT COMMENT GENERATION** is carried out under the guidance of **Mr. Yellaiah Ponnam**, Assistant Professor, **Department of Information Technology, Lords Institute of Engineering and Technology, Hyderabad,** and is submitted in partial fulfillment of the requirements for the award of the degree of **Bachelor of Engineering** in **Information Technology.**

This is a record of bonafide work carried out by us and the results embodied in this project have not been reproduced or copied from any source. The results embodied in this project report have not been submitted to any other university or institute for the award of any other degree or diploma.

Mohammed Omer Farooq (160923737048)

Mohammed Zeshan Mohiuddin (160923737053)

ACKNOWLEDGEMENT

We are very pleased to present this report of our project. This period of our student life has been truly rewarding, and a number of people were of immense help to us during the course of our research and the preparation of our report.

First, we wish to thank **God Almighty**, who created the heavens and the earth, who helped us in completing this project, and we also thank our parents who encouraged us in this period of research.

We would like to express our sincere gratitude to **The Management** of **Lords Institute of Engineering and Technology, Hyderabad** for providing us with an excellent platform and resources to successfully carry out this project.

We would also like to thank **Dr. Ravi Kishore Singh, Principal, L.I.E.T.** for extending his help in overall completion of this project work.

We would like to express our deep sense of gratitude to **Dr. K. Nagi Reddy, Professor & HOD, Information Technology, Lords Institute of Engineering & Technology, Hyderabad,** for his encouragement and cooperation throughout the project.

We would like to thank **Mr. Yellaiah Ponnam**, Assistant Professor, **Dept. of Information Technology, Lords Institute of Engineering & Technology, Hyderabad**, our project internal guide, for his guidance and help. His insight during the course of our research and regular guidance were invaluable to us.

Lastly, we acknowledge the support of our faculty members, friends, and family, who inspired and encouraged us throughout this journey.

Mohammed Omer Farooq (160923737048)
Mohammed Zeshan Mohiuddin (160923737053)

LORDS INSTITUTE OF ENGINEERING & TECHNOLOGY

Approved by AICTE/Affiliated to Osmania University/Estd.2003.

Accredited 'A' grade by NAAC

(UGC Autonomous Institution)

Vision of the Institute

Lords Institute of Engineering and Technology strives for excellence in professional education through quality, innovation and teamwork and aims to emerge as a premier institute in the state and across the nation.

Mission of the Institute

- To impart quality professional education that meets the needs of present and emerging technological world.
- To strive for student achievement and success, preparing them for life, career and leadership.
- To provide a scholarly and vibrant learning environment that enables faculty, staff and students to achieve personal and professional growth.
- To contribute to advancement of knowledge, in both fundamental and applied areas of engineering and technology.
- To forge mutually beneficial relationships with government organizations, industries, society and the alumni.

PRINCIPAL
Principal
Lords Institute of Engineering & Tech.
(An Autonomous Institution)
Sy. No. 32, Himayath Sagar, Hyderabad-500091.T.S.

LORDS INSTITUTE OF ENGINEERING & TECHNOLOGY

Approved by AICTE/Affiliated to Osmania University/Estd.2003.

Accredited 'A' grade by NAAC

(UGC Autonomous Institution)

DEPARTMENT OF INFORMATION TECHNOLOGY

Vision of the Department:

To be a frontier in Information Technology focusing on quality professional education, innovation and employability.

Mission of the Department:

DM1: Adopt innovative teaching learning processes to achieve stated outcomes and sustained performance levels.

DM2: Provide learner centric environment to enrich knowledge, skills and innovation through industry academia collaborations.

DM3: Train the concerned stakeholders on latest technologies to meet the industry needs.

DM4: Inculcate leadership, professional, interpersonal skills and ethical values to address the contemporary societal issues.

Note: DM: Department Mission

Head of the Department

Head of the Department
Information Technology
Lords Institute of Engineering & Technology
Himayath Sagar, Hyderabad-91

LORDS INSTITUTE OF ENGINEERING & TECHNOLOGY

Approved by AICTE/Affiliated to Osmania University/Estd.2003.
Accredited 'A' grade by NAAC
(UGC Autonomous Institution)
DEPARTMENT OF INFORMATION TECHNOLOGY

B.E Information Technology Program Educational Objectives (PEOs):

PEO1	Be a competent software engineer in IT and allied industry providing viable solutions.
PEO2	Exhibit professionalism, engage in lifelong learning and adapt to changing industry and societal needs.
PEO3	Be effective as an individual and also cohesive in a group to execute multidisciplinary projects.
PEO4	Pursue higher studies in Information Technology with emphasis on competency and innovation.

Head of the Department
Head of the Department
Information Technology
Lords Institute of Engineering & Technolo
Himayath Sagar, Hyderabad-91

LORDS INSTITUTE OF ENGINEERING & TECHNOLOGY

Approved by AICTE/Affiliated to Osmania University/Estd.2003.

Accredited 'A' grade by NAAC

(UGC Autonomous Institution)

DEPARTMENT OF INFORMATION TECHNOLOGY

B.E Information Technology Program Outcomes (POs):

Engineering Graduates will be able to:

S. No.	Program Outcomes (POs)
1.	**PO1: Engineering Knowledge:** Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop the solution of complex engineering problems.
2.	**PO2: Problem Analysis:** Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)
3.	**PO3: Design/Development of Solutions:** Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)
4.	**PO4: Conduct Investigations of Complex Problems:** Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).
5.	**PO5: Engineering Tool Usage:** Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)
6.	**PO6: The Engineer and The World:** Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).

S. No.	Program Outcomes (POs)
7.	**PO7: Ethics:** Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)
8.	**PO8: Individual and Collaborative Team work:** Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
9.	**PO9: Communication:** Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences
10.	**PO10: Project Management and Finance:** Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.
11.	**PO11: Life-Long Learning:** Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

Head of the Department
Head of the Department
Information Technology
Lords Institute of Engineering & Technology
Himayath Sagar, Hyderabad-91

LORDS INSTITUTE OF ENGINEERING & TECHNOLOGY

Approved by AICTE/Affiliated to Osmania University/Estd.2003.

Accredited 'A' grade by NAAC

(UGC Autonomous Institution)

DEPARTMENT OF INFORMATION TECHNOLOGY

B.E Information Technology Program Specific Outcomes (PSOs):

PSO1	**Professional Skills:** Identify, analyze requirements, design and implement computer programs in the area of artificial intelligence, machine learning, big data analytics and networking of varying complexity.
PSO2	**Problem-Solving Skills:** Deliver quality service to IT industry and provide solutions to real-life problems applying modern computer languages, software methods and tools.

Head of the Department
Head of the Department
Information Technology
Lords Institute of Engineering & Technolo
Himayath Sagar, Hyderabad-91



LORDS INSTITUTE OF ENGINEERING & TECHNOLOGY

Approved by AICTE/Affiliated to Osmania University/Estd.2003.

Accredited 'A' grade by NAAC

(UGC Autonomous Institution)

DEPARTMENT OF INFORMATION TECHNOLOGY

Course Outcomes: Project Phase-I

After completion of the project, the student will be able to

S.No	Description	Bloom's Taxonomy Level
PP.I.CO1	Demonstrate the ability to synthesize and apply the knowledge and skills acquired in the academic program to the real-world problems.	BTL3
PP.I.CO2	Evaluate different solutions based on economic and technical feasibility	BTL4
PP.I.CO3	Effectively plan a project and confidently perform all aspects of project management.	BTL5
PP.I.CO4	Demonstrate effective written and oral communication skills	BTL4
PP.I.CO5	Prepare the documentation report and perform the presentation of the project work	BTL3

LORDS INSTITUTE OF ENGINEERING & TECHNOLOGY

Approved by AICTE/Affiliated to Osmania University/Estd.2003.

Accredited 'A' grade by NAAC

(UGC Autonomous Institution)

DEPARTMENT OF INFORMATION TECHNOLOGY

Course Articulation Matrix:

Mapping of Course Outcomes (COs) with Program Outcomes (POs) and Program Specific Outcomes (PSOs):

Course Outcomes (COs)	Program Outcomes (POs)											Program Specific Outcomes (PSOs)	
	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO 10	PO 11	PSO1	PSO2
PP.I.CO1	3	2	2	-	2	2	-	-	3	-	-	2	2
PP.I.CO2	3	3	2	3	2	-	2	-	3	-	-	-	2
PP.I.CO3	3	3	3	-	3	3	1	2	2	3	2	3	-
PP.I.CO4	3	2	2	3	3	3	3	2	-	-	2	3	3
PP.I.CO5	3	3	3	3	3	3	3	2	3	3	-	3	3
AVG	**3**	**2.6**	**2.4**	**3**	**2.6**	**2.7**	**2.2**	**2**	**2.7**	**3**	**2**	**2.7**	**2.5**

Level:

1- Low correlation (Low), 2- Medium correlation (Medium), 3-High correlation (High)

SDG Mapping:

SDG	Mapped Indicator	SDG	Mapped Indicator	SDG	Mapped Indicator
1 NO POVERTY		7 AFFORDABLE AND CLEAN ENERGY		13 CLIMATE ACTION	
2 ZERO HUNGER		8 DECENT WORK AND ECONOMIC GROWTH		14 LIFE BELOW WATER	
3 GOOD HEALTH AND WELL-BEING		9 INDUSTRY, INNOVATION AND INFRASTRUCTURE	✓	15 LIFE ON LAND	
4 QUALITY EDUCATION	✓	10 REDUCED INEQUALITIES		16 PEACE, JUSTICE AND STRONG INSTITUTIONS	
5 GENDER EQUALITY		11 SUSTAINABLE CITIES AND COMMUNITIES		17 PARTNERSHIPS FOR THE GOALS	
6 CLEAN WATER AND SANITATION		12 RESPONSIBLE CONSUMPTION AND PRODUCTION			

ABSTRACT

The increasing demand for programming education and the growing size of classes necessitate immediate and personalized feedback. However, integrating Large Language Models (LLMs), such as ChatGPT, into introductory programming courses raises concerns about AI-assisted cheating. In large-scale educational settings, faulty code submissions may cause LLMs to perform unnecessary analyses, resulting in excessive token consumption.

This report presents a GPT-4o-based code review system designed to provide accurate feedback while reducing token usage and mitigating AI-assisted cheating. Unlike general-purpose LLM tools designed for professional developers, the proposed system is pedagogically tailored to primary and secondary school students, emphasizing the necessity of code reviews and the delivery of learner-friendly feedback. The system consists of a Code Review Module (CRM), which reduces token consumption through a Review Necessity Chain (RNC), and a Code Correctness Check Module (CCM), which combines test-case validation with LLM-based assessment. To discourage AI-assisted cheating, the system automatically provides feedback on submitted code without directly revealing the correct answers. Correct solutions can be accessed only through the “Ask Code Tutor” button. The research paper on which this work is based reported that, in usability testing, such a system detected up to 42.86% more errors than a conventional online judge, that over 80% of its generated reviews were semantically aligned with human feedback (BERTScore), and that it achieved a blocking success rate of 86% against answer-seeking prompts. These figures belong to that source paper and are not results of this project. They are the targets that motivate the design, which aims to provide more accurate feedback than conventional automated code review systems while improving token efficiency and supporting self-directed learning through educational feedback.

CONTENTS

Certificate i
Declaration ii
Acknowledgement iii
Vision and Mission of the Institute iv
Vision and Mission of the Department v
Program Educational Objectives vi
Program Outcomes vii
Program Specific Outcomes ix
Course Outcomes x
Course Articulation Matrix xi
Abstract xii

CHAPTER-1 INTRODUCTION 1
1.1 Overview 1
1.2 Background 2
1.3 Problem Statement 3
1.4 Objectives of the Project 3
1.5 Scope of the Project 4
1.6 Organisation of the Report 4

CHAPTER-2 LITERATURE SURVEY 6

CHAPTER-3 SYSTEM ANALYSIS AND REQUIREMENTS 11
3.1 Existing System 11
3.2 Proposed System 11
3.3 Description of the Modules 12
3.4 System Architecture 13
3.5 Process Model Used with Justification 14
3.6 Software Requirement Specification 16

CHAPTER-4 SYSTEM STUDY AND DESIGN 19
4.1 Feasibility Study 19
4.2 Methodology 20
4.3 Algorithms and Models Used 21
4.4 System Design 24

CHAPTER-5 INPUT AND OUTPUT DESIGN 28
5.1 Input Design 28
5.2 Output Design 30
5.3 User Interface Design 30
5.4 Processing of a Submission 31

CHAPTER-6 SOFTWARE ENVIRONMENT **32**
6.1 Introduction 32
6.2 Programming Language 32
6.3 Artificial Intelligence Model 32
6.4 Supporting Libraries and Tools 33
6.5 Dataset Environment 34
6.6 Proposed Software Requirements 34
6.7 Hardware Requirements 35
6.8 Proposed System Workflow 36
6.9 Summary 36
CHAPTER-7 SYSTEM TESTING **37**
7.1 Introduction 37
7.2 Objectives of System Testing 37
7.3 Testing Methodology 38
7.4 Proposed Test Cases 39
7.5 Evaluation Metrics 40
7.6 Results Discussion 41
7.7 Limitations and Challenges 42
7.8 Future Testing and Improvement 43
7.9 Conclusion 43
CONCLUSION **44**
FUTURE SCOPE 45
REFERENCES **46**

List of Figures

Sl. No.	Fig. No.	Name of the Figure	Page No
1	3.1	System architecture of Code Mentor GPT	14
2	3.2	SDLC (umbrella) model followed in the project	14
3	4.1	Workflow of Code Mentor GPT	20
4	4.2	Transformer encoder-decoder architecture	21
5	4.3	BERT architecture	22
6	4.4	Architecture and working of GPT-4o	23
7	4.5	Use case diagram	24
8	4.6	Class diagram	25
9	4.7	Sequence diagram	26
10	4.8	Deployment diagram	26
11	4.9	Data flow diagram	27
12	5.1	Sample rows of the code review dataset	30

List of Tables

Sl. No.	Table No.	Name of the Table	Page No
1	3.1	Functional requirements	16
2	3.2	Hardware and software requirements	18
3	5.1	Routes of the Flask application	31
4	6.1	Proposed software requirements	35
5	6.2	Minimum hardware requirements	35
6	7.1	Proposed test cases	39

CHAPTER-1

INTRODUCTION

1.1 Overview

Programming is no longer a subject kept for university classrooms. Many countries have brought coding into primary and secondary schools, and national education policies now describe digital and computational skills as part of basic competence [1], [2], [3], [4], [5]. As more students begin to write programs at an early age, the question of how their work is checked and explained to them becomes more important than it was when programming was taught only to small groups of older learners.

A beginner who submits a program wants to know more than whether it passed or failed. Online judges, which are now the usual way of marking programming exercises, report a verdict such as accepted, wrong answer or time limit exceeded, and judges built for school students, such as the one described in [6], follow the same pattern. That is useful to an experienced programmer, but a child who has just learned loops and conditions often cannot tell from a verdict alone what went wrong or how to think about the fix. A teacher can explain it, but a teacher with forty or fifty students cannot read and comment on every submission the same day. Students therefore wait for feedback, and by the time it arrives they have often moved on to the next topic.

Large Language Models (LLMs) have changed what is possible here. A model such as GPT-4o can read a piece of source code and describe, in plain language, what it does, where it is weak and how it might be improved. Studies of ChatGPT-style models report that they can write feedback that is detailed and fluent, and that in some respects it compares well with feedback written by instructors [7]. For a teacher this is attractive, because comments that once took minutes to write can be produced in seconds.

The same ability creates new problems when the model is used directly in a classroom. A general-purpose chatbot is built to be helpful, so when a student pastes in a faulty program it will usually return a corrected one. The student gets a working answer without understanding the mistake, which defeats the purpose of the exercise and raises the concern of AI-assisted cheating discussed in [8], [9] and [10]. A second problem is cost. LLM services are charged by the number of tokens processed, and a class of beginners submits a great many programs that are incomplete, empty or plainly wrong. Sending every one of them to the model for a full review wastes tokens and, in a large course, wastes money.

Code Mentor GPT: A Hybrid AI Framework for Educational Code Review and Intelligent Comment Generation is the project reported here. It is built around a GPT-4o based review system that tries to answer these problems together. It gives learner-friendly comments without handing out the finished solution, it decides whether a submission deserves a detailed review before spending tokens on it, and it checks the program against test cases before commenting. An extension component adds explanatory comments to the submitted source code so that the learner can follow what each part is doing.

1.2 Background

Automated assessment of programs has a long history. The earliest systems compiled a submission, ran it against a fixed set of inputs and compared the output with the expected output. Later work tried to say something more useful than pass or fail. Singh, Gulwani and Solar-Lezama, for example, proposed a way of generating feedback for introductory programming assignments automatically by finding the small changes that would make a student's program correct [11]. Such rule-based and search-based approaches work well within the problems they were designed for, but each new assignment needs new rules or a new reference solution.

Transformer-based language models removed that restriction. Trained on very large collections of text and source code, they can handle many different problems without being told the rules of each one [12]. This has produced a quick growth in research on LLMs for programming education, which Amos et al. have mapped in a bibliometric study that also lists the risks of dependency, plagiarism and weaker critical thinking [13]. Classroom tools such as CodeAid have tried to balance the needs of students and educators by limiting what the assistant is allowed to show [14], and other studies have tested GPT models for feedback on high-school programming assignments [15], tiered formative feedback [16] and code review in undergraduate computer science programmes [17], [18].

A recent review of 90 LLM-based programming support systems makes the same point from another side: the systems share similar teaching goals but differ widely in how they limit the help they give, and they are rarely described in a way that lets them be compared. How a tool restricts its help is therefore a design decision that has to be made on purpose, and in an educational code review tool it is the central one. The system proposed in this report restricts help in a simple way. Review comments are produced

automatically and never contain the complete answer, and a corrected solution can only be requested through a separate “Ask Code Tutor” option.

The work in this report follows the research paper by D.-K. Lee on a GPT-based code review system for programming language learning. That paper reports a usability test in which the system detected up to 42.86% more errors than a conventional online judge, a BERTScore analysis in which more than 80% of the generated reviews were semantically aligned with human feedback, and a blocking success rate of 86% against answer-seeking prompts. These figures belong to the source paper. They are quoted in this report as the targets that motivate the design, and they are not presented as results of our own experiments.

1.3 Problem Statement

Growing class sizes make it impractical for instructors to give every learner immediate and personal feedback on programming exercises. Conventional online judges only report whether a program is correct. General-purpose LLM tools can explain errors, but they often reveal the full solution, which encourages AI-assisted cheating, and they analyse faulty submissions in full, which consumes tokens without adding educational value. There is also no structured check of whether a program is correct before the feedback is written, so the comments a learner receives can be inconsistent or unrelated to the real fault.

The problem addressed by this project is therefore to design and build a code review system that gives accurate, learner-friendly feedback on submitted programs, does so with fewer tokens, and does not reveal the answer.

1.4 Objectives of the Project

The objectives of the project are as follows.

1. To develop a GPT-4o based automated code review system that gives meaningful and learner-friendly feedback on student-submitted source code without revealing complete solutions.
2. To collect and preprocess a code review dataset containing programming code and the corresponding review comments, and to use it for developing and evaluating the system.
3. To implement a Code Review Module (CRM) with a Review Necessity Chain (RNC) that identifies which submissions need a detailed review, so that unnecessary processing and token consumption are reduced.

4. To develop a Code Correctness Check Module (CCM) that validates a submitted program against test cases before review comments are generated, so that the feedback is based on the actual correctness of the code.
5. To implement a Feedback Generation Module that points out coding mistakes, suggests improvements and good practice, and gives educational guidance without giving direct solutions.
6. To develop an automatic code comment generation extension that studies elements such as classes, functions and variables and writes comments explaining their purpose, and to provide a Flask based web interface through which a learner can upload code and view the generated review and comments.

1.5 Scope of the Project

The system is meant for learners at primary and secondary school level, where students write short programs and need explanations in simple language. The prototype built in this phase works on source code files submitted through a web page. The comment generation extension is written for Python source code, and the review model is evaluated on a code review dataset taken from Kaggle, which holds code patches together with human-written review comments.

The scope of the first phase covers the design of the full framework, the preparation of the dataset, the working of the review generation and comment generation components, the Flask interface, and a first check of review quality using BERTScore. A full comparison with other systems, a measurement of the tokens saved by the Review Necessity Chain, and classroom trials with real students are outside this phase and are listed in the future scope.

1.6 Organisation of the Report

The report is arranged in seven chapters followed by the conclusion. Chapter 1 introduces the problem, the background and the objectives. Chapter 2 surveys the literature on LLM-based feedback, code review automation and academic integrity. Chapter 3 analyses the existing system, describes the proposed system and its modules, and gives the software requirement specification. Chapter 4 covers the feasibility study, the methodology, the algorithms used and the design of the system with the UML and data flow diagrams. Chapter 5 describes the input and output design and the user interface. Chapter 6 describes the software environment and the libraries and tools used.

Chapter 7 reports the testing carried out and discusses the results obtained. The report ends with the conclusion, the future scope and the list of references.

The overall organisation of the report provides a systematic understanding of the proposed CODE MENTOR GPT: A Hybrid AI Framework for Educational Code Review and Intelligent Comment Generation. The chapters progress from identifying the problem and reviewing existing research to analysing the proposed framework, explaining its design, and discussing the evaluation approach. Particular emphasis is placed on the use of Large Language Models (LLMs) for generating meaningful code feedback, identifying potential programming errors, and improving code comprehension through intelligent comment generation. The report also considers the importance of feedback accuracy, educational usefulness, and academic integrity in AI-assisted programming education. This structured presentation establishes the theoretical foundation and design specifications required for the subsequent development and evaluation of the proposed framework.

CHAPTER-2

LITERATURE SURVEY

2.1: Literature Survey

1. "Natural Language Generation and Understanding of Big Code for AI-Assisted Programming: A Review"

Man Fai Wong, Shangxin Guo, Ching Nam Hang, Siu Wai Ho, Chee Wei Tan, 2023

This paper provides a comprehensive review of the literature concerning the utilization of Natural Language Processing (NLP) techniques, with a particular focus on transformer-based large language models (LLMs) trained using Big Code, within the domain of AI-assisted programming tasks. LLMs, augmented with software naturalness, have played a crucial role in facilitating AI-assisted programming applications, including code generation, code completion, code translation, code refinement, code summarization, defect detection, and clone detection. Notable examples of such applications include the GitHub Copilot powered by OpenAI's Codex and DeepMind AlphaCode. This paper presents an overview of the major LLMs and their applications in downstream tasks related to AI-assisted programming. Furthermore, it explores the challenges and opportunities associated with incorporating NLP techniques with software naturalness in these applications, with a discussion on extending AI-assisted programming capabilities to Apple's Xcode for mobile software development. This paper also presents the challenges of and opportunities for incorporating NLP techniques with software naturalness, empowering developers with advanced coding assistance and streamlining the software development process.

2. "Empowering education with AI: Addressing ethical concerns"

Sara Saylam, Nilgun Duman, Yetkin Yildirim, Kseniya Satsevich, 2023

There has been a rapid advancement of technology in the realm of education, and artificial intelligence (AI) has become just one of the many tools utilized by members of educational institutions. However, with the swift integration of AI into the education system, many ethical challenges and dilemmas have surfaced; primarily driven by students' misuse of the transformative technology. The potential impact on students' critical thinking skills, autonomy, and ethical decision-making further highlights the urgency to address these issues. This article explores the detrimental effects resulting from the unethical use of AI, along with proposing significant policies and guidelines in

order to maximize the beneficial utilization of AI within educational institutions. Additionally, a comprehensive analysis of relevant studies will be presented to sustain the argument stated and contribute to the development of an AI learning environment that enables the prospering of both students and faculty.

3. "Can Large Language Models Provide Feedback to Students? A Case Study on ChatGPT"

Wei Dai, Jionghao Lin, Hua Jin, Tongguang Li, Yi-Shan Tsai, Dragan Gašević, 2023

Educational feedback has been widely acknowledged as an effective approach to improving student learning. However, scaling effective practices can be laborious and costly, which motivated researchers to work on automated feedback systems (AFS). Inspired by the recent advancements in the pre-trained language models (e.g., ChatGPT), we posit that such models might advance the existing knowledge of textual feedback generation in AFS because of their capability to offer natural-sounding and detailed responses. Therefore, we aimed to investigate the feasibility of using ChatGPT to provide students with feedback to help them learn better. Our results show that i) ChatGPT is capable of generating more detailed feedback that fluently and coherently summarizes students' performance than human instructors; ii) ChatGPT achieved high agreement with the instructor when assessing the topic of students' assignments; and iii) ChatGPT could provide feedback on the process of students completing the task, which might benefit students developing learning skills.

4. "LLaMA-Reviewer: Advancing Code Review Automation with Large Language Models through Parameter-Efficient Fine-Tuning"

Junyi Lu, Lei Yu, Xiaojia Li, Li Yang, Chun Zuo, 2023

The automation of code review activities, a long-standing pursuit in software engineering, has been primarily addressed by numerous domain-specific pre-trained models. Despite their success, these models frequently demand extensive resources for pre-training from scratch. In contrast, Large Language Models (LLMs) provide an intriguing alternative, given their remarkable capabilities when supplemented with domain-specific knowledge. However, their potential for automating code review tasks remains largely unexplored. In response to this research gap, we present LLaMA-Reviewer, an innovative framework that leverages the capabilities of LLaMA, a popular LLM, in the realm of code review. Mindful of resource constraints, this framework employs parameter-efficient fine-tuning (PEFT) methods, delivering high performance

while using less than 1% of trainable parameters. An extensive evaluation of LLaMA-Reviewer is conducted on two diverse, publicly available datasets. Notably, even with the smallest LLaMA base model consisting of 6.7B parameters and a limited number of tuning epochs, LLaMA-Reviewer equals the performance of existing code-review-focused models. The ablation experiments provide insights into the influence of various fine-tuning process components, including input representation, instruction tuning, and different PEFT methods. To foster continuous progress in this field, the code and all PEFT-weight plugins have been made open-source.

5. "Exploring Organization of Subject Curriculum for Improvement of Coherence between General Guidelines and Subject Curriculum"

Jiyoung Yoon, Jungduk Ohn, 2024

Objectives The purpose of this study is to propose the organization of subject curriculum to support a learner-centered curriculum design and practice for competency development, while ensuring coherence between general guidelines and subject curriculum. Methods Analyzing the teaching, learning, and assessment guidelines of the 2022 revised curriculum framework, we derived the key points that should be considered in the design of teachers' teaching and learning activities. In light of this, documents on the Canada Ontario curriculum (Language, Mathematics, SSHG, Science and Technology) were analyzed. Based on this, we analyzed and summarized the organization of subject curriculum to help design teaching and learning. Results The 2022 revised curriculum framework emphasized 'connection within and between subjects, connection with life, learning about inquiry methods, active participation of students, and assessment as learning'. The subject curriculum of Ontario provides support for connection within and across subjects, application of learning content in real-world contexts, guidance on procedural knowledge and thinking processes, posing essential questions related to learning content, and supporting students' reflection on learning through curriculum expectations, strands, concepts and big ideas, processes, assessment and evaluation of student achievement, and optional teacher supports. Conclusions Based on these results, it was confirmed that it is possible to enhance coherence between general guidelines and subject curriculum and support teachers' curriculum design and implementation through the establishment of a curriculum document framework. In the national curriculum development, it was suggested that there is a need for a curriculum design that systematizes content elements within the categories of process and skills, presents performance standards, and demonstrates the

connection of knowledge within and across subjects according to the characteristics of the curriculum.

6. “Designing LLM-Resistant Programming Assignments: Insights and Strategies for CS Educators”

Bradley McDanel, Ed Novak, 2025

The rapid advancement of Large Language Models (LLMs) like ChatGPT has raised concerns among computer science educators about how programming assignments should be adapted. This paper explores the capabilities of LLMs (GPT-3.5, GPT-4, and Claude Sonnet) in solving complete, multi-part CS homework assignments from the SIGCSE Nifty Assignments list. Through qualitative and quantitative analysis, we found that LLM performance varied significantly across different assignments and models, with Claude Sonnet consistently outperforming the others. The presence of starter code and test cases improved performance for advanced LLMs, while certain assignments, particularly those involving visual elements, proved challenging for all models. LLMs often disregarded assignment requirements, produced subtly incorrect code, and struggled with context-specific tasks. Based on these findings, we propose strategies for designing LLM-resistant assignments. Our work provides insights for instructors to evaluate and adapt their assignments in the age of AI, balancing the potential benefits of LLMs as learning tools with the need to ensure genuine student engagement and learning.

7. “A Bibliometric Exposition and Review on Leveraging LLMs for Programming Education”

Joanah Pwanedo Amos, Oluwatosin Ahmed Amodu, Raja Azlina Raja Mahmood, Akanbi Bolakale Abdulqudus, Anies Faziehan Zakaria, Abimbola Rhoda Iyanda, 2025

The world is experiencing an AI revolution, with large language models (LLMs) transforming various industries, including education. Academics are striving to harness the potential of LLMs while also contending with their risks. This paper presents the first bibliometric analysis focused on LLM research in programming education, identifying leading countries, authors, and institutions while analyzing key terms and popular keywords in this field. Additionally, it highlights influential studies on topics such as introductory programming, computer science, computing, programming education, and prompt engineering, discussing key insights from these works. Findings indicate that LLMs could play a significant role in programming education and may be

integrated into computer science curricula. However, careful consideration is needed to ensure their benefits outweigh their risks across various use cases. This study specifically examines ChatGPT as a representative LLM, exploring its benefits and limitations as both a learning aid for students and a support tool for professionals. It also evaluates the quality of ChatGPT-generated code and its effectiveness in simplifying programming concepts for beginners. Furthermore, the ethical implications of increasing reliance on LLMs for programming tasks, including concerns about dependency, plagiarism, and potential effects on critical thinking, are addressed. By contributing to the ongoing discourse on integrating AI tools like ChatGPT in programming education, this research emphasizes the importance of responsible and ethical usage to maximize benefits for students, educators, and the broader educational community.

8. "Exploring the Design Space of LLM-Based Programming Support in CS Education: A Scoping Review through the Lens of Assistance Governance"

Minsun Kim, S. Moonwara A. Monisha, Zihan Wu, David H. Smith IV, 2026

As large language models (LLMs) become integrated into programming education, learner-facing systems increasingly differ in how that assistance is bounded, enacted, and controlled. These governance decisions are often described implicitly, making it difficult to compare systems in educationally meaningful ways. To address this gap, we conduct a scoping review and qualitative synthesis of 90 peer-reviewed LLM-based programming support systems in CS education. We analyze assistance governance through three dimensions, which we refer to collectively as PEA: Policy, capturing what forms of help are allowed or restricted; Enforcement, capturing how those boundaries are operationalized through interaction and system behavior; and Authority, capturing who can configure, adapt, or override them during use. Our findings show that systems often share similar pedagogical goals, but implement those goals through varied enforcement mechanisms. At the same time, authority remains highly centralized in system logic, with fewer systems giving learners or instructors runtime control. This work contributes PEA as a three-dimensional analytic lens, a governance codebook empirically refined within these dimensions, and a map of underexplored configurations in the current design space of LLM-based programming support. By making these explicit and comparable, PEA offers a vocabulary for analyzing existing systems and designing future tools that are pedagogically bounded, configurable, and accountable.

CHAPTER-3

SYSTEM ANALYSIS AND REQUIREMENTS

3.1 Existing System

Programming courses at school level mostly depend on three kinds of tools: conventional code evaluation tools, online judges and general-purpose large language models. The first two are good at finding compilation errors, syntax mistakes and failed test cases. What they usually return, however, is either very little information, such as a verdict and the number of the failed test, or, in the case of an LLM, a complete corrected program. Neither of these helps a beginner to understand the mistake.

Giving a full solution encourages copying and takes away the effort of solving the problem, which is the very skill the exercise is meant to build. Many LLM-based systems also send the whole source code to the model without deciding which parts need attention, so tokens are consumed on every submission, including those that are empty, meaningless or clearly broken. Finally, these systems do not have a structured way of confirming that a program is correct before the feedback is written. As a result learners get comments that vary in quality, explanations they did not need, or direct answers in place of constructive review, and the quality of learning suffers.

Disadvantages of the existing system

1. Complete solutions are generated, which may encourage cheating.
2. A large number of tokens is consumed during the review.
3. Feedback is often inconsistent and unfocused.
4. There is no structured validation of code correctness.
5. Learners' independent problem-solving skills are reduced.

3.2 Proposed System

The proposed system is a GPT-4o based code review system designed for programming education. Its main aim is to give accurate feedback without revealing the complete solution. It is built from two cooperating modules. The Code Review Module (CRM) uses a Review Necessity Chain (RNC) to decide which review points matter, so that unnecessary token use is reduced and the review stays focused. The Code Correctness Check Module (CCM) validates the submitted program against test cases before any feedback is produced.

Instead of replacing the student's program with a corrected one, the system offers suggestions that help the learner find and repair the mistake. The quality of the

generated reviews is measured with BERTScore, by comparing the reviews produced by the model with the reference reviews in the dataset. An extension component adds automatic explanatory comments to the submitted code. Together these features aim to improve learning, reduce the chance of cheating, lower the computing cost and give guidance that is useful for a learner.

Advantages of the proposed system

1. Educational feedback is generated without revealing complete answers.
2. Token consumption is reduced through the Review Necessity Chain (RNC).
3. Code is validated by the Code Correctness Check Module (CCM) before review.
4. Cheating is discouraged and self-directed learning is encouraged.
5. Review quality is measured with BERTScore.

3.3 Description of the Modules

The system is divided into the following modules, together with an extension module for automatic comments.

3.3.1 Dataset Collection Module

This module collects programming code samples and their review comments from the Kaggle code review dataset. Each record contains the code patch, the name of the programming language, a prompt that also includes the code, and the review written for it. The data is preprocessed so that code and review form pairs which can be used to evaluate the system. In the present prototype the first 3,000 records of the dataset file are loaded.

3.3.2 GPT-4o Code Review Module (CRM)

The module receives the learner's source code and studies its quality with GPT-4o. Through the Review Necessity Chain it concentrates on the review points that really matter and produces useful comments without exposing a complete solution. Submissions that do not need a detailed review can be filtered out before the expensive analysis is carried out.

3.3.3 Code Correctness Check Module (CCM)

This module checks whether the submitted program meets the required test cases and verifies its correctness before review comments are generated. Combining the test result with the model's own assessment makes the feedback more accurate and more relevant to the actual fault in the program.

3.3.4 Feedback Generation Module

The module writes the review comments that the learner finally sees. The comments point out coding mistakes, improvements, good practice and possible optimisation. They are written so that they guide the learner without giving the answer directly.

3.3.5 Performance Evaluation Module

This module judges the quality of the generated reviews using BERTScore. The predicted review is compared with the reference review from the dataset, and a higher score indicates a closer semantic match and so a more accurate review.

3.3.6 Extension: Automatic Code Comment Generation

The extension studies elements of the source code such as classes, functions and variables and inserts comments that explain their purpose. In the implementation this is done by a separate Python module, named PythonComment, whose generate_comments_advanced function takes the source text and returns it with comments added. The comments make it easier for a learner to follow what each section of a program does.

3.4 System Architecture

Figure 3.1 shows the architecture of the system. A learner logs in to the web interface and uploads a source-code file. The file is passed to the review component, which tokenises the code and generates a review with the language model. The same file is passed to the comment generation extension. The submitted code, the generated review and the commented code are then displayed together on one page. During development and evaluation, the notebook loads the dataset and compares the generated reviews with the reference reviews using BERTScore.

The system architecture of CODE MENTOR GPT consists of several interconnected modules that work together to provide intelligent code review and comment generation. The learner uploads source code through the web interface, which is then processed by the code review component. The language model analyses the submitted code and generates feedback on potential errors, code quality, and readability. In parallel, the PythonComment module generates explanatory comments for relevant code sections. The generated review and commented code are displayed to the learner through the interface. The evaluation module compares generated reviews with reference reviews using BERTScore. This modular architecture supports efficient processing, clear feedback presentation, and future system enhancements.

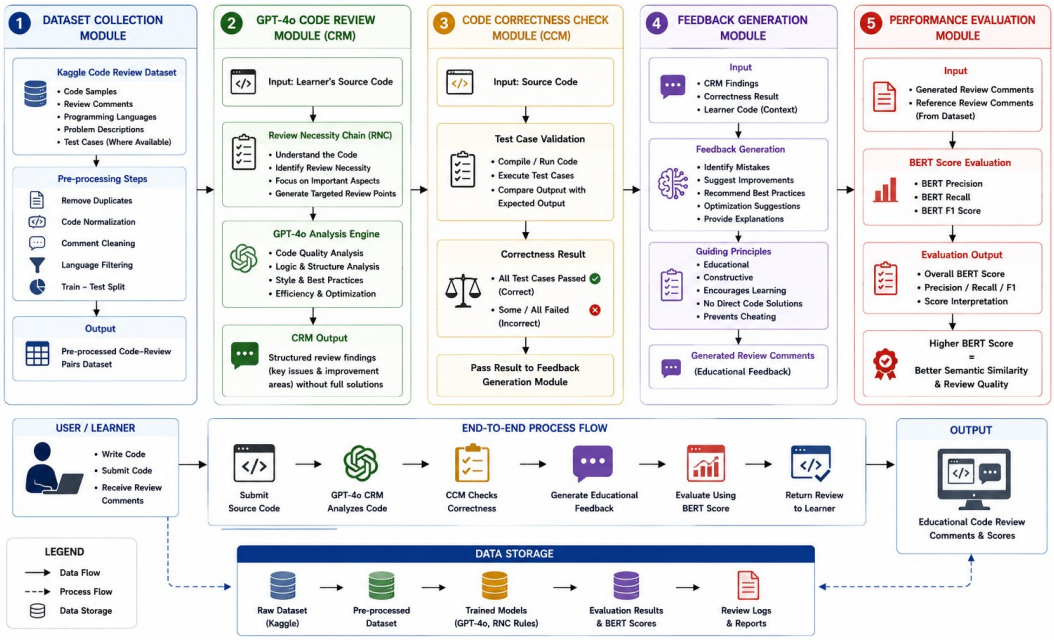


Fig. 3.1: System architecture of Code Mentor GPT

3.5 Process Model Used with Justification

The project follows the Software Development Life Cycle (SDLC) in its umbrella form. SDLC is the standard set of stages used in the software industry to plan, build, test and maintain software. The umbrella form is chosen because maintenance, which covers all later changes to the software, does not end at a fixed point but continues over the whole life of the system, like the frame of an umbrella that covers every other stage. This suits Code Mentor GPT, where prompts, test cases and models are likely to be revised as feedback from learners comes in. The stages are the following.

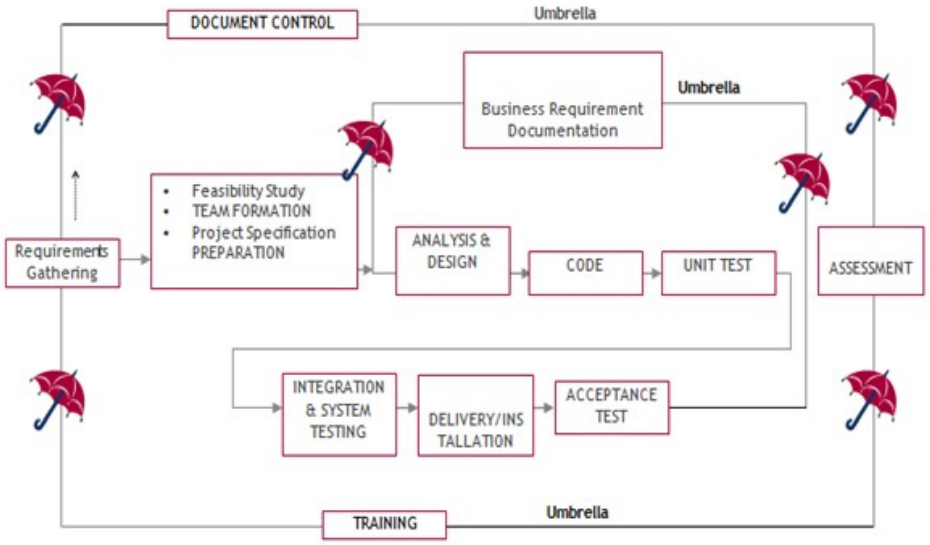


Fig. 3.2: SDLC (umbrella) model followed in the project

Requirement gathering

The goals of the project were identified first and then broken down into individual requirements, each with a title and a short description. For this project they include the generation of review comments without answers, the validation of code with test cases, the reduction of token use and the evaluation of review quality. The requirements are listed in Section 3.6. A requirements traceability matrix links every goal to its requirements, and later to design elements, code and test cases, so that nothing specified is left unbuilt or untested.

Analysis

In the analysis stage the feasibility, the risks and the technical approach were studied. The main risks identified were the cost of sending every submission to a large model, the possibility of the model revealing answers, and the difficulty of judging the quality of free-text reviews. These led to the Review Necessity Chain, the answer-restricting design and the BERTScore evaluation. The outputs of this stage were the project plan and the schedule for the next stages.

Design

The design stage turned the requirements into design elements: the architecture, the UML diagrams, the data flow diagrams and the layout of the web pages. These are presented in Chapter 4 and Chapter 5. Each design element is linked to the requirement it satisfies.

Coding

In the coding stage the design was implemented in Python. The notebook files handle model loading, review generation and evaluation, and the Flask application provides the web interface. Test inputs were prepared along with the code. The software environment used is described in Chapter 6.

Integration and testing

The components were then combined and run together in a test setup. The test cases listed in Chapter 7 were executed to confirm that the dataset loads, that reviews and comments are generated, and that the results are displayed correctly in the web page.

Installation and acceptance

In the last stage the software is installed on the target machine and checked against the original requirements before it is accepted by the user, in this case the project guide and the department. The software environment is described in Chapter 6.

Maintenance

Maintenance includes correcting faults, improving prompts and test cases, and adapting the system to new datasets or models. Because it never has a final end point, the SDLC is drawn in the form of an umbrella.

3.6 Software Requirement Specification

3.6.1 Overall description

A Software Requirements Specification (SRS) describes the behaviour expected of a system before it is built. It lists the interactions users will have with the software, usually as use cases, and also the non-functional requirements, which are the constraints on design and implementation such as performance, quality standards and design limits. Requirements are commonly divided into business requirements, which state in business terms what must be delivered, product requirements, which describe the properties of the system, and process requirements, which describe the activities of the team that develops it. The requirements for Code Mentor GPT are given below.

3.6.2 Functional requirements

ID	Requirement
FR1	The system shall let a user log in before using the code review facility.
FR2	The system shall accept a source-code file uploaded by the user.
FR3	The system shall generate a review of the submitted code using the GPT-4o based review model.
FR4	The generated review shall point out mistakes and improvements without giving the complete solution.
FR5	The system shall check the submitted code against test cases before review comments are generated (CCM).
FR6	The system shall decide through the Review Necessity Chain whether a detailed review is required (CRM).
FR7	The system shall generate explanatory comments for the submitted source code (extension).
FR8	The system shall display the submitted code, the generated review and the commented code on one page.
FR9	The system shall evaluate generated reviews against reference reviews using BERTScore.

ID	Requirement
FR10	The system shall let the user log out and return to the home page.

Table 3.1: Functional requirements

3.6.3 Non-functional requirements

- **Usability:** the interface should be simple enough for a school learner or a teacher to use without training.
- **Performance:** a review should be returned within a reasonable waiting time, and token consumption should be kept as low as possible.
- **Educational integrity:** automatic feedback must not reveal the complete solution.
- **Reliability:** the system should handle an uploaded file without failing, and should return a clear message for invalid input.
- **Portability:** the application should run on a standard Windows machine with Python installed.
- **Maintainability:** the modules should be separate so that the model, the prompts or the test cases can be changed without rewriting the rest.

3.6.4 External interface requirements

User Interface: The system provides a web-based interface through which learners can upload source-code files, view code review feedback, and access automatically generated comments. Jupyter Notebook is used for model loading, dataset processing, and evaluation.

Hardware Interface: The system operates on a standard computer or laptop with a keyboard, display, sufficient memory, and processing capacity. No specialised hardware is required.

Software Interface: Python is used for implementing the core modules, while Flask supports the web interface. Required libraries support code processing, language model integration, and BERTScore evaluation.

Communication Interface: The application communicates with the browser using HTTP or HTTPS. API communication is used when interacting with an external language model service.

Data Interface: The system accepts source code as input and produces code review feedback and commented code as output. Reference datasets are used to evaluate generated reviews using BERTScore.

3.6.5 Hardware and software requirements

Item	Specification
Processor	Intel i3 (minimum)
Hard disk	500 GB
RAM	4 GB (minimum)
Operating system	Windows 10 (minimum)
Programming language	Python 3.10.11

Table 3.2: Hardware and software requirements

CHAPTER-4

SYSTEM STUDY AND DESIGN

4.1 Feasibility Study

A feasibility study is carried out at the start of a project to find out whether the system can be built and used with the time, money and skills available. It looks at three aspects, which are economic, operational and technical feasibility. Any system can be called feasible if resources and time are unlimited, so the purpose of the study is to check the project against realistic limits.

4.1.1 Economic feasibility

A system must be a good investment as well as technically possible. For this project the development cost is small. Python, Flask, Jupyter and the supporting libraries are free and open source, the dataset is publicly available on Kaggle, and development runs on an ordinary computer. The main running cost of an LLM based tool is the cost of tokens, and this is exactly what the Review Necessity Chain is designed to reduce by filtering submissions that do not need a detailed review. On this basis the system is considered economically feasible.

4.1.2 Operational feasibility

A system is useful only if its intended users can work with it. The users here are learners and teachers, and the interface asks for very little: log in, choose a file and read the result. Teachers are likely to welcome a tool that removes repeated review work, and learners receive feedback sooner than they would from a person. Since the feedback is written to guide and not to give the answer, the tool fits the way programming is already taught and there is no reason to expect resistance to its use.

4.1.3 Technical feasibility

The technology needed is available and well documented. Transformer language models can be loaded through the Hugging Face transformers library, BERTScore is available as a Python package, and Flask is enough for a web interface of this size. The prototype has been built and run on a standard Windows machine with Python 3.10.11. The system is therefore technically feasible, with the practical note that running a large language model locally needs more memory than the minimum configuration listed in Chapter 3.

4.2 Methodology

The working of the system can be described in four steps, which follow the order in which a submission is handled.

1. **Collection and preprocessing.** Code-review pairs are collected from the dataset and preprocessed so that the code and the reference review can be used for evaluation.
2. **Validation.** The submitted code is validated against test cases by the Code Correctness Check Module, so that the review is based on the real state of the program.
3. **Selection of review points.** The Review Necessity Chain is applied to find the important review points and to avoid analysing the whole program when that is not needed.
4. **Feedback generation.** GPT-4o generates educational feedback that explains the problems and the possible improvements without giving the finished solution.

The reviews are finally compared with the reference reviews by BERTScore, and the extension adds explanatory comments to the code shown to the learner. Figure 4.1 shows the flow of work.

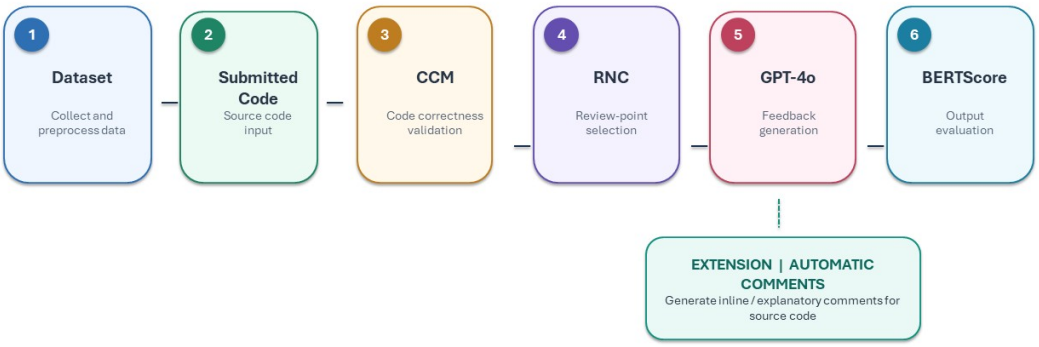


Fig. 4.1: Workflow of Code Mentor GPT

4.3 Algorithms and Models Used

4.3.1 Transformer

The Transformer is a neural network architecture introduced by Vaswani et al. in 2017 in the paper “Attention Is All You Need”. It replaced the recurrent networks that earlier language models were built on. A recurrent network (RNN) reads a sentence one word at a time, so information from early words fades by the time the end is reached, and an LSTM reduces this problem without removing the need to process words in sequence. In addition, the static word embeddings used with such networks give a word the same vector in every context, although a word such as “point” means different things in “a sharp point” and “to point at people”. The Transformer reads the whole sequence at once and uses self-attention to decide how much each word should influence every other word, so the meaning of a word can be worked out from its surroundings.

Positional encodings are added to the input embeddings because the model has no built-in sense of word order. The central operation is scaled dot-product attention, in which each token produces a query vector Q, a key vector K and a value vector V. The attention output is

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}(Q K^{T} / \sqrt{d_k}) V \qquad (4.1)$$

where d_k is the dimension of the key vectors. Multi-head attention applies this operation several times in parallel with different learned projections and joins the results, so that the model can attend to different kinds of relationship at once. Each layer also contains a position-wise feed-forward network. A full Transformer has an encoder, which turns the input sequence into a representation, and a decoder, which produces the output sequence from it.

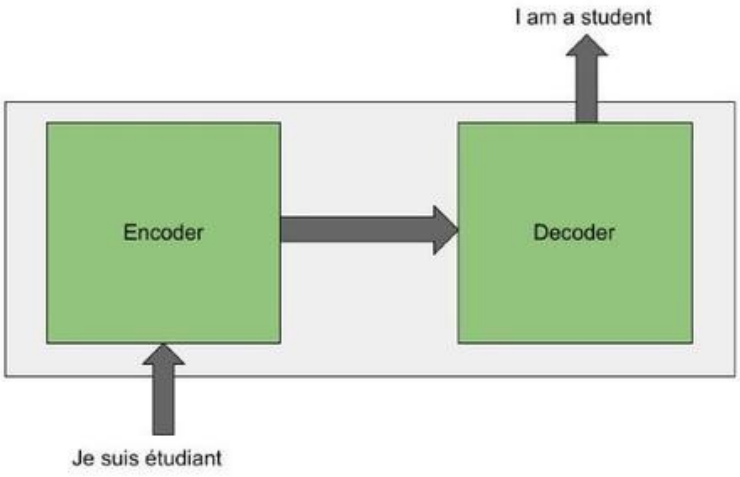


Fig. 4.2: Transformer encoder-decoder architecture

4.3.2 BERT

BERT (Bidirectional Encoder Representations from Transformers) is a model built from the encoder part of the Transformer. Older models read text in one direction, but BERT reads it from both directions at once, so each word is understood using the words on its left and its right. It is first pre-trained on large amounts of text with two tasks, Masked Language Modelling and Next Sentence Prediction, and then fine-tuned for specific tasks such as sentiment analysis and text classification.

The input is split into tokens, and special tokens are added: [CLS] at the beginning and [SEP] to separate sentences. Each token is converted into an embedding that combines the token itself, its position and the segment it belongs to. These embeddings pass through a stack of encoder layers in which self-attention gives the model a weighted view of the other tokens in the sentence. In this project BERT is not used to produce reviews. It supplies the contextual embeddings on which BERTScore is calculated.

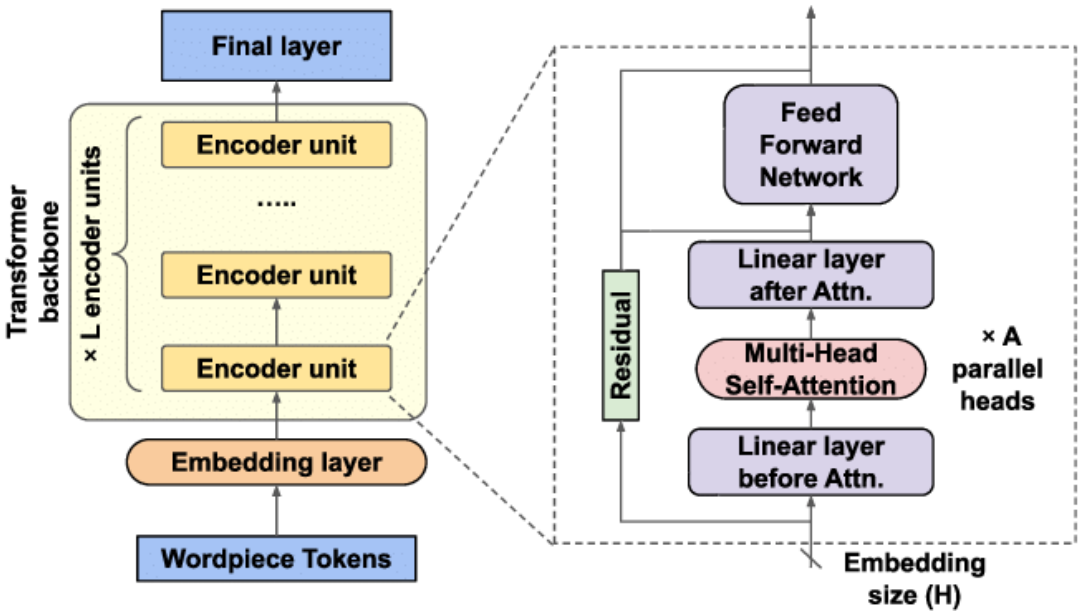


Fig. 4.3: BERT architecture

4.3.3 GPT-4o

GPT-4o is a large language model that follows a decoder-only Transformer design. It can read natural language, program code and instructions and produce text in reply. In programming education it can read the code written by a student and comment on errors, structure and style. Unlike a rule-based analyser, it takes the context of the whole program into account when it writes a response.

The model works on tokens. The input, which may be a question or a piece of source code, is divided into tokens by a tokenizer, and the tokens are converted into

embeddings with position information. The embeddings go through many Transformer layers where self-attention finds which earlier tokens are relevant to the one being produced and feed-forward layers transform the result. An output layer then gives a probability for each possible next token, one token is selected, and it is appended to the sequence and used as part of the input for the next step. Because each token depends only on those before it, the process is called autoregressive, and it continues until the response is complete.

For code review the same process is applied to source code. The submitted program is tokenised and embedded, attention relates statements, variables and functions to one another, and the generated tokens are converted back into readable text that is shown to the learner as feedback.

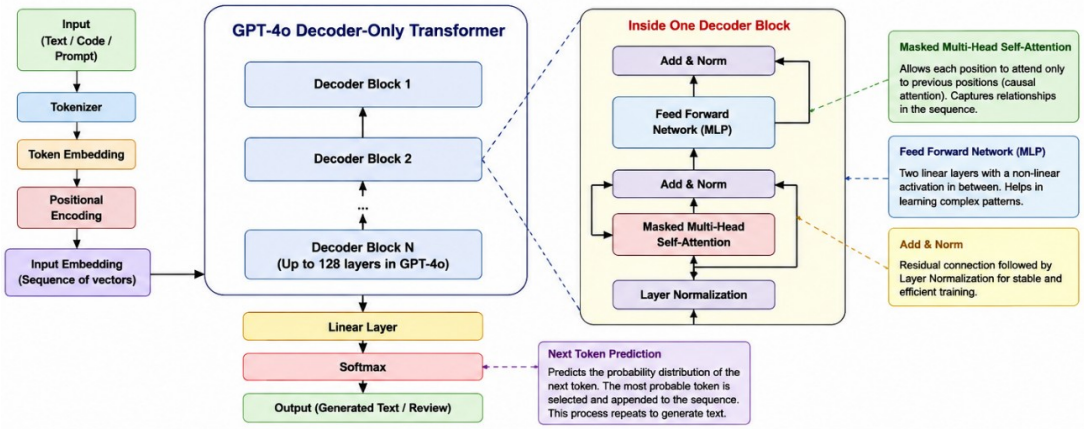


Fig. 4.4: Architecture and working of GPT-4o

4.3.4 BERTScore

BERTScore [19] compares a candidate text with a reference text by finding, for each token, the most similar token in the other text according to cosine similarity of their contextual embeddings. If x is the reference with tokens x_i and $\hat{x}$ is the generated text with tokens $\hat{x}$_j, recall, precision and the F1 score are

$$R = (1/|x|)\ \Sigma\ \max(\ x_i^{T}\ \hat{x}_j\ ) \quad \text{over } x_i \in x \tag{4.2}$$

$$P = (1/|\hat{x}|)\ \Sigma\ \max(\ x_i^{T}\ \hat{x}_j\ ) \quad \text{over } \hat{x}_j \in \hat{x} \tag{4.3}$$

$$F1 = 2 \cdot P \cdot R\ /\ (\ P + R\ ) \tag{4.4}$$

where the maximum is taken over the tokens of the other text and the embeddings are normalised. The implementation used here sets the scorer to use the bert-base-uncased model for English and applies baseline rescaling, which spreads the scores over a more readable range. A higher value shows that the generated review is semantically closer to the reference review.

4.4 System Design

The Unified Modelling Language (UML) is used to describe the design of the system. UML is a standard notation for presenting an analysis model, with rules for syntax and meaning. A UML model describes a system from several views: the user view, which shows the system from the standpoint of its users, the structural view, which shows the static structure, the behavioural view, which shows how the parts interact, the implementation view, and the environmental view, which shows the surroundings in which the system runs. The diagrams used in this report are described below.

4.4.1 Use case diagram

A use case diagram shows who uses the system and what they can do with it. In Code Mentor GPT the main actor is the learner, who can log in, upload a source-code file, view the generated review, view the automatically commented code and log out. The model, the dataset and the evaluation component work inside the system and are not actors.

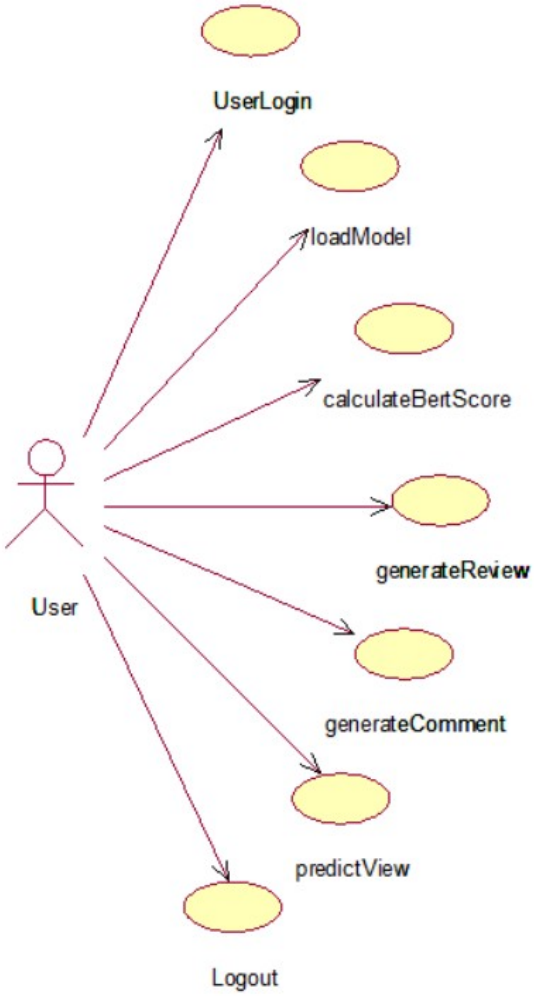


Fig. 4.5: Use case diagram

4.4.2 Class diagram

A class diagram shows the main classes of the system, with the name at the top of each box, the attributes in the middle and the operations at the bottom. For this system the main elements are the web application, which handles the pages and the upload, the review generator, which wraps the tokenizer and the language model, the comment generator, which wraps the PythonComment module, the dataset and the evaluator that computes BERTScore.

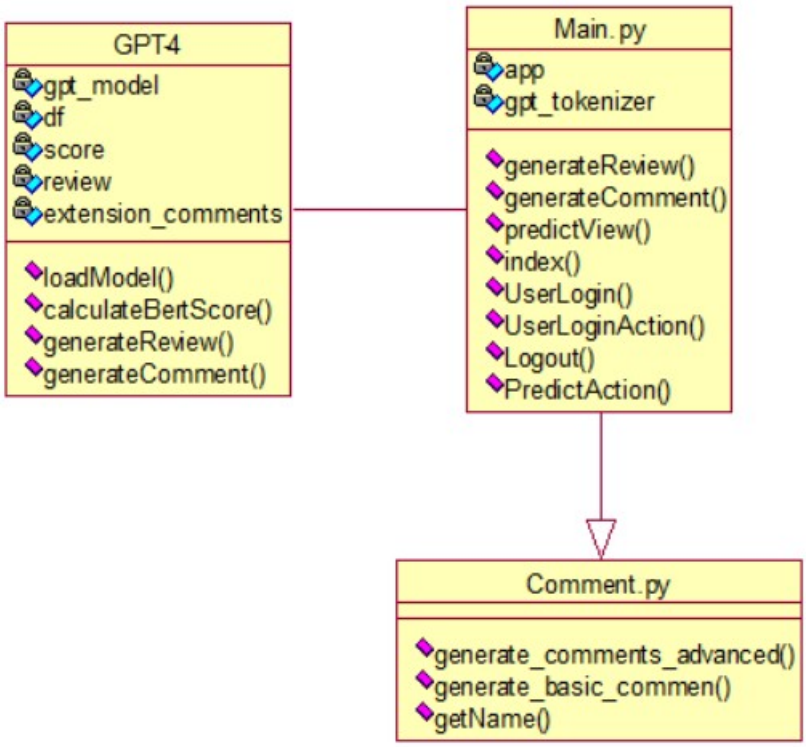


Fig. 4.6: Class diagram

4.4.3 Sequence diagram

A sequence diagram shows the messages that pass between the parts of the system in time order. When a learner uploads a file, the browser sends it to the Flask route, the file is saved, the review generator tokenises the code and asks the model for a review, the comment generator produces the commented code, and the page returns the three results to the browser.

The sequence diagram also represents the order of requests and responses exchanged between the system components. It helps identify the responsibilities of each module and illustrates how the code review and comment generation processes operate together. This interaction ensures that the learner receives both feedback and explanatory comments in a structured manner.

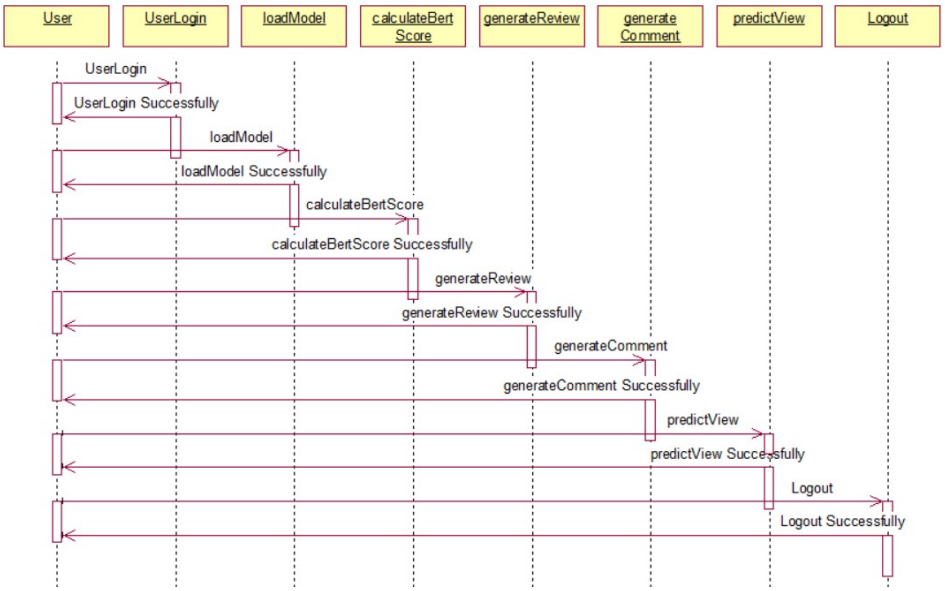


Fig. 4.7: Sequence diagram

4.4.4 Deployment diagram

A deployment diagram shows the physical nodes on which the software runs. In the prototype a single computer hosts the Flask web server, the language model and the dataset, and the learner reaches the application through a web browser at the local address of the server.

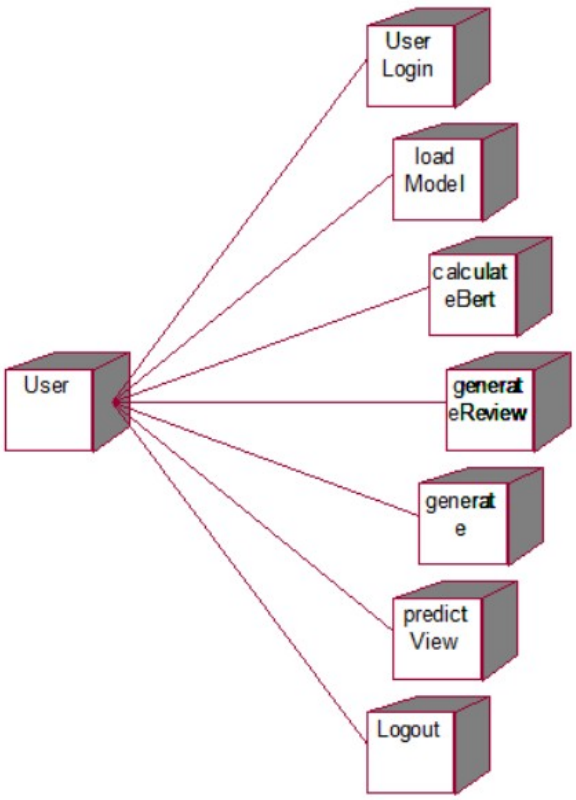


Fig. 4.8: Deployment diagram

4.4.5 Data flow diagram

A data flow diagram (DFD) shows how data moves through a system in terms of inputs, processes and outputs. At the top level the learner supplies source code to the system and receives a review and a commented version of the code. At the next level the code passes through validation, review generation and comment generation, and the dataset supplies reference reviews to the evaluation process.

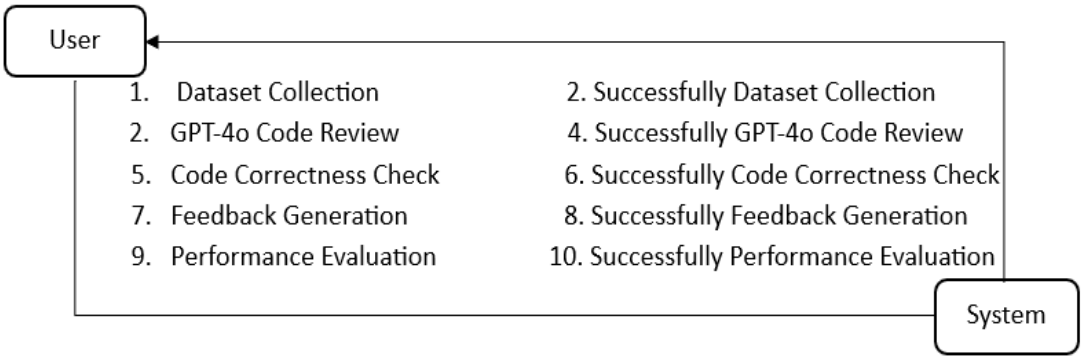


Fig. 4.9: Data flow diagram

CHAPTER-5

INPUT AND OUTPUT DESIGN

5.1 Input Design

Input design defines the information accepted by the system, the format in which it is provided, and the procedures followed to process it. An effective input design ensures that the system receives appropriate data, reduces input errors, and simplifies interaction for the user. In CODE MENTOR GPT, input design plays an important role in collecting authentication details, accepting source-code files for review, and loading datasets for experimental evaluation. The system uses three primary types of input: login details, source-code files, and evaluation datasets. Each input serves a specific purpose and is processed by the corresponding system component.

5.1.1 Login Details

The login page accepts the learner's username and password before granting access to the code review functionality. These fields provide a basic authentication mechanism and prevent users from directly accessing the review page without entering the required credentials. The username identifies the account, while the password is used to verify the login attempt.

In the current prototype, authentication is implemented using a single predefined account rather than a database of registered users. When the learner submits the login form, the application compares the entered credentials with the configured values. If the credentials match, access to the review page is granted. Otherwise, the system displays the message "Invalid login details" and allows the learner to try again. The login form is designed to accept text input for the username and concealed input for the password. Integration with a user database, password hashing, and support for multiple accounts are considered possible future enhancements.

5.1.2 Source-Code File

The source-code file is the primary input for the automated code review and comment generation processes. After successful login, the learner selects a source-code file through the file upload interface and submits it for analysis. The uploaded file contains the program that needs to be examined for potential errors, readability issues, and opportunities for improvement.

When the file is submitted, the application receives the uploaded content and temporarily stores it in the application's static folder. A new upload replaces the

previously stored file, allowing the prototype to process one submission at a time. The stored file is subsequently read as text and passed to the code review component. The review generator prepares the source code for analysis and sends the appropriate input to the language model to generate feedback. In parallel, the comment generation module processes the code to produce explanatory comments for relevant programming constructs, such as functions, classes, and variables.

The system then presents the submitted code, the generated review, and the commented source code through the web interface. The source-code input is therefore central to the complete workflow, as both the review generator and comment generation module depend on the uploaded program. In a production environment, additional input validation, file-size restrictions, supported file extensions, and secure temporary storage would help improve reliability and security.

5.1.3 Dataset

The dataset input is used during the development and experimental evaluation of CODE MENTOR GPT. The project uses a CSV file containing code review examples obtained from the Kaggle code review dataset. The dataset is stored in the Dataset folder and loaded into the development environment using Python libraries for data processing.

The dataset contains fields representing source-code changes, programming language information, prompts, and reference reviews. The patch column contains code changes, the language column identifies the programming language, the prompt column provides the code-related input used in the review process, and the response column contains the reference review written by a human reviewer. These fields provide the information required to prepare inputs for the model and compare its generated feedback with reference responses.

For the experiments, the first 3,000 rows are loaded from the dataset. The selected records are processed and supplied to the review generation workflow as required. The generated reviews are then compared with the corresponding reference reviews using BERTScore to estimate their semantic similarity. This evaluation helps assess whether the generated feedback conveys information similar to that found in human-written reviews. The dataset is used for development and evaluation rather than being entered manually by the learner through the web interface.

Proper dataset handling is important to ensure that the records are loaded correctly and that the source-code inputs and reference reviews remain associated with one another. The dataset can also support future experiments involving different sample sizes,

programming languages, and evaluation methods to examine the performance of the proposed framework under varied conditions.

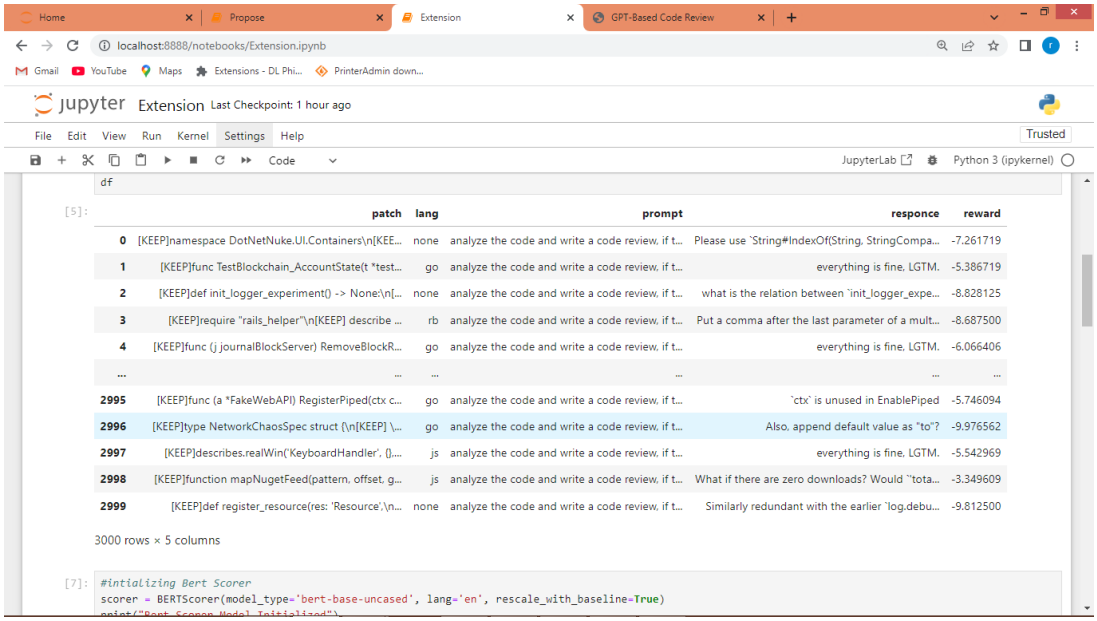


Fig. 5.1: Sample rows of the code review dataset

5.2 Output Design

Output design determines how results are presented. The system produces two kinds of output.

Web output. After a file is uploaded the page shows a table with three columns. The first column holds the submitted code, the second holds the review comment generated by the GPT-4o based model, and the third holds the code with the comments produced by the extension. When the submitted code is good, the review simply says that everything is fine. When it can be improved, the review suggests a refinement, and the third column shows the code with explanatory comments written above its parts.

Notebook output. In the notebook the output is the BERTScore of a generated review, the generated review and the extension comments for a test file. These outputs are used while developing and checking the system.

5.3 User Interface Design

The web interface has four pages. The home page, reached at the address http://127.0.0.1:5000/index when the server is running on the local machine, has a link to the user login page. The login page asks for the user name and password. After login the user screen shows a welcome message and the link to GPT code review generation. The review page has the file chooser and the submit button, and the result table is shown

on the same screen after submission. The pages are served by routes defined in the Flask application, which are listed in Table 5.1.

Table 5.1: Routes of the Flask application

Route	Method	Purpose
/index	GET, POST	Displays the home page
/UserLogin	GET, POST	Displays the login page
/UserLoginAction	GET, POST	Checks the login details and opens the user screen or shows an error
/Predict	GET, POST	Displays the page for uploading a code file
/PredictAction	GET, POST	Receives the file, generates the review and the comments and shows the result table
/Logout	GET	Returns the user to the home page

5.4 Processing of a Submission

When the learner presses the submit button, the application first removes any earlier uploaded file and saves the new one. The generateReview function reads the file, appends a marker so that the text ends with “Comment:”, tokenises the combined text with a maximum length of 1,024 tokens and asks the model to generate a continuation. The generated tokens are decoded to text and returned as the review. The generateComment function passes the same file to the PythonComment module. The three results are placed in the table and returned to the page.

The generation settings are a maximum length of 1,024 tokens, sampling turned on with a temperature of 0.7, and the end-of-text token used for padding. The temperature adds some variety, which means that two reviews of the same code may not be word-for-word identical.

CHAPTER-6

SOFTWARE ENVIRONMENT

6.1 Introduction

The software environment consists of the programming languages, libraries, frameworks, development tools, and supporting resources required for the development of the proposed system. The project, Code Mentor GPT: A Hybrid AI Framework for Educational Code Review and Intelligent Comment Generation, aims to provide automated code review and educational feedback to programming learners.

The proposed system combines a GPT-4o-based Code Review Module (CRM), a Review Necessity Chain (RNC), a Code Correctness Check Module (CCM), and a Feedback Generation Module. It also includes an extension for generating comments that explain programming elements such as classes, functions, and variables.

The software environment is designed to support dataset preparation, language-model integration, code analysis, feedback generation, and performance evaluation. Jupyter Notebook is considered for experimentation and evaluation, while Flask is proposed as the web framework for the user interface.

6.2 Programming Language

6.2.1 Python

Python is the primary programming language specified for the proposed system. It provides a wide range of libraries for artificial intelligence, natural language processing, data processing, and web application development.

Python is intended to support the processing of submitted source code, integration of the GPT-4o model, execution of the code-review workflow, and generation of educational feedback. It can also be used to implement the supporting modules and evaluation procedures.

The supporting project document specifies Python 3.10.11 as the software requirement. The exact Python version used for the final implementation should be confirmed and documented during development.

6.3 Artificial Intelligence Model

6.3.1 GPT-4o

GPT-4o is the language model identified for generating code reviews and educational feedback. It is intended to analyze submitted source code and produce explanations

concerning programming mistakes, code quality, coding practices, and possible improvements.

Unlike a conventional code-checking system that primarily reports compilation errors or failed test cases, the proposed framework aims to provide explanations that help learners understand the reasons behind programming problems.

The model is intended to operate within a structured review workflow. The system should provide guidance rather than automatically revealing a complete corrected program in every response. This approach is intended to encourage independent problem-solving and reduce unnecessary dependence on AI-generated solutions.

6.4 Supporting Libraries and Tools

6.4.1 Jupyter Notebook

Jupyter Notebook provides an interactive environment for developing and examining Python code. It can be used for dataset inspection, experimentation with the code-review workflow, model-related operations, and evaluation of generated reviews.

The supporting document describes Jupyter Notebook as the environment used for model generation, loading, and evaluation. Separate notebooks can be maintained for the proposed method and its extension, depending on the final implementation.

6.4.2 Flask Web Framework

Flask is a lightweight Python web framework intended to provide the web interface for the proposed system. It can connect the user interface with the backend code-review functionality.

The planned interface allows a learner to log in, upload a source-code file, view the submitted program, and examine the generated review. The extension is intended to display automatically generated comments alongside relevant sections of the source code.

6.4.3 Pandas and NumPy

Pandas can be used to load and organize the structured code-review dataset, inspect its columns, and prepare the required data for experimentation. NumPy provides numerical operations that may be useful in data processing and evaluation.

Their use should be aligned with the actual preprocessing and evaluation code included in the project.

6.4.4 Transformers and Datasets

The Transformers and Datasets libraries can support model-related experimentation and dataset management where required by the selected implementation. They may be used to load model components, process training or evaluation examples, and manage the data supplied to the model workflow.

The precise role of these libraries depends on whether the final system uses local model components, a hosted model interface, or a combination of both.

6.4.5 BERTScore

BERTScore is a metric used to assess the semantic similarity between generated text and reference text. In the proposed system, it can be used to compare generated code-review comments with reference reviews from the dataset.

BERTScore can provide useful evidence about the similarity of generated feedback to reference comments. However, it does not independently establish whether a review correctly identifies a programming error. Technical correctness and educational usefulness require additional evaluation.

6.5 Dataset Environment

The proposed framework uses a code-review dataset containing programming code and associated review information. The supporting document describes the following dataset fields:

- patch: Contains the code or code changes being reviewed.
- lang: Identifies the programming language associated with the code.
- prompt: Contains the prompt or code-related input supplied for review.
- response: Contains the corresponding review or suggestion.

The dataset is intended to provide examples for experimentation and comparison between generated reviews and reference comments. Before use, the dataset should be inspected to identify missing values, duplicate records, inconsistent formats, and unsuitable examples.

The preprocessing procedure should be documented according to the actual operations performed. The number of records used for development and evaluation should also be recorded so that the experimental process can be reproduced.

6.6 Proposed Software Requirements

The following table summarizes the software requirements described or indicated in the project documents.

Component	Software or Tool	Purpose
Operating system	Windows 10 or later	Development environment
Programming language	Python	Backend logic and code processing
Language model	GPT-4o	Code review and educational feedback
Notebook environment	Jupyter Notebook	Experimentation and evaluation
Web framework	Flask	User interface and backend integration
Data processing	Pandas and NumPy	Dataset inspection and processing
Model-related tools	Transformers and Datasets, where required	Model and dataset workflows
Evaluation metric	BERTScore	Semantic comparison of reviews
Dataset	Code-review dataset	Source-code and review examples

Table 6.1: Proposed software requirements

The exact package versions and installation requirements should be finalized after confirming the implementation environment.

6.7 Hardware Requirements

The supporting document identifies the following minimum hardware configuration.

Hardware Component	Requirement
Processor	Intel Core i3 or equivalent
RAM	4 GB minimum
Storage	500 GB hard disk
Operating system	Windows 10 minimum

Table 6.2: Minimum hardware requirements

These specifications are the requirements listed in the supporting document. Actual resource requirements may be higher depending on the language-model access method, dataset size, and the number of simultaneous users. If GPT-4o is accessed through a

hosted service, local hardware requirements will differ from those needed to run a large model locally.

6.8 Proposed System Workflow

The software environment supports the following overall workflow:

1. Dataset preparation: Load and inspect the code-review dataset and prepare the required code-review examples.
2. Code submission: Allow the learner to submit a source-code file through the proposed Flask interface.
3. Review necessity assessment: Apply the Review Necessity Chain (RNC) to determine whether detailed review is required.
4. Code correctness checking: Use the Code Correctness Check Module (CCM) to assess the submitted program against appropriate test cases.
5. Review generation: Use GPT-4o to generate relevant suggestions and explanations.
6. Educational feedback: Present feedback that identifies potential problems and guides the learner toward improvements.
7. Automatic comment generation: Generate explanatory comments for relevant code elements as part of the extension.
8. Performance evaluation: Compare generated reviews with reference reviews using BERTScore and other suitable checks.

The exact sequence and interaction of the modules must be verified against the implemented system**.**

6.9 Summary

This chapter describes the proposed software environment for Code Mentor GPT. Python, GPT-4o, Jupyter Notebook, Flask, the code-review dataset, and BERTScore form the principal elements of the planned development and evaluation workflow. The framework combines review-necessity assessment, code correctness checking, educational feedback generation, and automatic code commenting. These components will need to be integrated and evaluated to establish the effectiveness of the proposed approach.

CHAPTER-7

SYSTEM TESTING

7.1 Introduction

System testing is an important stage in software development because it examines whether the system behaves according to its requirements. For Code Mentor GPT, testing is intended to evaluate the handling of source-code submissions, the generation of code reviews, the operation of correctness checking, the quality of educational feedback, and the evaluation of generated comments.

The proposed system combines a GPT-4o-based Code Review Module (CRM), a Review Necessity Chain (RNC), a Code Correctness Check Module (CCM), and a Feedback Generation Module. Therefore, testing must consider both the individual modules and their interaction within the complete workflow.

As this report describes the Phase-I research and design stage, the following sections establish the testing methodology and expected outcomes. Actual test results should be recorded only after the corresponding tests have been executed and verified.

7.2 Objectives of System Testing

The principal objectives of testing are:

1. To examine whether the dataset can be loaded and processed correctly.
2. To evaluate whether the proposed system generates relevant code-review comments.
3. To assess whether the correctness-checking component can identify errors using appropriate test cases.
4. To examine whether the generated feedback is understandable and useful to programming learners.
5. To evaluate the quality of generated reviews using BERTScore and suitable complementary methods.
6. To verify the interaction between the review, correctness-checking, and feedback-generation components.
7. To identify limitations and areas requiring improvement before further development.

7.3 Testing Methodology

7.3.1 Unit Testing

Unit testing examines individual components independently to determine whether they perform their intended functions.

For Code Mentor GPT, the planned unit tests cover dataset loading, code-input processing, review generation, correctness checking, feedback generation, and evaluation. Each component should be supplied with suitable inputs, and its output should be checked against the expected behaviour.

Invalid input, missing data, and unexpected responses should also be considered to assess error handling.

7.3.2 Integration Testing

Integration testing examines whether individual modules communicate and operate together correctly.

In the proposed framework, integration testing should examine the interaction between the Review Necessity Chain, Code Correctness Check Module, GPT-4o review generation, and feedback presentation. It should also verify the movement of data from the user interface to the backend and the return of generated feedback to the interface.

Any errors caused by inconsistent data formats, missing responses, or incorrect module connections should be recorded and addressed.

7.3.3 Functional Testing

Functional testing evaluates whether the system meets its specified requirements. The proposed test inputs should include valid programs, programs with syntax errors, programs with known logical errors, and programs that produce unexpected output.

The resulting feedback should be examined to determine whether it relates to the submitted code and whether it provides appropriate explanations or suggestions.

A program should not be classified as correct solely because the language model reports no errors. Where possible, execution results and predefined test cases should be used to verify program behaviour.

7.3.4 Performance Evaluation

Performance evaluation examines the quality of the generated reviews and the efficiency of the review process.

BERTScore may be used to compare generated reviews with reference comments from the dataset. Additional measurements may include response time, error-detection

performance, review relevance, and the number of cases in which unnecessary detailed review can be avoided.

Each metric should be calculated using a documented dataset, evaluation procedure, and set of test conditions.

7.4 Proposed Test Cases

The following table defines the test cases planned for evaluating the proposed system.

Test Case ID	Test Case	Test Procedure	Expected Outcome
TC-01	Dataset Loading	Load the code-review dataset and inspect its required fields	The dataset is read successfully, and the required fields are available
TC-02	Code Input	Submit a valid source-code file	The submitted code is received and processed
TC-03	Code Review Generation	Submit a program containing a known issue	The system attempts to generate relevant review feedback
TC-04	Code Correctness Check	Test a program against predefined test cases	The system records whether the program satisfies the selected test cases
TC-05	Review Necessity Chain	Supply examples requiring different levels of review	The RNC follows its defined review-selection logic
TC-06	Feedback Generation	Submit code requiring an explanation or improvement suggestion	The generated feedback addresses the identified issue
TC-07	Automatic Comment Generation	Supply source code containing classes, functions, or variables	The extension attempts to generate relevant explanatory comments
TC-08	Semantic Evaluation	Compare generated reviews with reference reviews	BERTScore is calculated using the documented evaluation procedure
TC-09	Integration Testing	Execute the complete review workflow	The connected components exchange data as intended

Test Case ID	Test Case	Test Procedure	Expected Outcome
TC-10	Invalid Input Handling	Submit empty, malformed, or unsupported input	The system handles the input appropriately without an unhandled failure

Table 7.1: Proposed test cases

Testing status: These are proposed test cases. Their actual status must be updated after execution. No test should be marked as passed unless its outcome has been observed and documented**.**

7.5 Evaluation Metrics

7.5.1 BERTScore

BERTScore is proposed for comparing generated code-review comments with reference reviews. It uses contextual text representations to estimate semantic similarity.

The metric reports precision, recall, and F1-score. The F1-score combines precision and recall into a single measure of similarity between the generated and reference text.

A higher score can indicate greater semantic similarity, but it does not guarantee that the generated review is technically correct. For this reason, the results should be considered alongside manual review and appropriate program-verification methods.

7.5.2 Error-Detection Performance

Error-detection performance examines whether the system identifies known programming problems. Programs with predefined errors should be tested, and the generated findings should be compared with verified reference outcomes.

The evaluation should record correctly identified errors, missed errors, and incorrect error reports. This will help determine whether the framework provides useful feedback beyond the results available from conventional test-case checking.

7.5.3 Review Relevance

Review relevance measures how closely the generated feedback addresses the submitted program and its actual issues. The evaluation should consider whether each review is specific, understandable, and consistent with the code.

Feedback that contains unrelated suggestions or unsupported claims should be identified during this assessment.

7.5.4 Response Time

Response time measures the time required to process a code submission and return feedback. It should be measured over multiple test examples under documented conditions.

The model-access method, source-code length, network conditions, and processing configuration should be considered when interpreting the measurements.

7.5.5 Review Necessity and Token Efficiency

The Review Necessity Chain is intended to avoid unnecessary detailed reviews. Its effectiveness should be evaluated by recording which submissions require further review and comparing the selected cases with a defined reference procedure.

Any claim about reduced token consumption must be supported by actual token-usage measurements or another clearly documented method. The reduction should not be assumed solely from the presence of the RNC module.

7.6 Results Discussion

The supporting project document describes the intended evaluation of generated reviews using BERTScore and includes reported results from its associated study. However, those reported values should not automatically be presented as results produced by the current Phase-I project unless the corresponding experiments, data, and evaluation procedure have been verified.

At the current research stage, the following areas remain important for evaluation.

Code-review quality: The generated reviews should be examined to determine whether they identify relevant programming issues and provide explanations that learners can understand.

Correctness checking: The CCM should be evaluated using programs with known expected outputs and predefined test cases. This will help determine whether the system can distinguish program behaviour from unsupported language-model assumptions.

Educational feedback: The feedback should be assessed to determine whether it guides students toward understanding and correcting their mistakes rather than unnecessarily providing complete solutions.

Automatic comments: The comments generated for classes, functions, and variables should be examined for clarity, technical accuracy, and relevance to the corresponding source-code elements.

Semantic similarity: BERTScore can provide a quantitative comparison between generated reviews and reference comments. Its results should be interpreted together with qualitative assessment because semantically similar text can still contain technical errors.

Efficiency: The RNC workflow should be evaluated using recorded token usage and review-selection outcomes to determine whether it avoids unnecessary processing without omitting important issues.

The observations and measurements obtained from these evaluations will provide evidence about the strengths and limitations of the proposed framework. Until the experiments have been carried out and verified, numerical outcomes and successful test statuses should remain unreported.

7.7 Limitations and Challenges

Several limitations may affect the proposed system.

1. Language-model reliability: GPT-4o may generate incomplete, inaccurate, or unsupported code-review comments.
2. Logical error detection: Some errors depend on runtime behaviour, input conditions, or execution context and cannot be reliably identified from source-code inspection alone.
3. Reference-data quality: Missing, inconsistent, or low-quality reference comments may affect evaluation results.
4. Semantic evaluation: BERTScore measures semantic similarity and cannot independently verify programming correctness.
5. Token efficiency: The effectiveness of the RNC must be established through recorded measurements.
6. Educational usefulness: Feedback should be appropriate for the learner's level and should support independent problem-solving.
7. System integration: The interface, model workflow, correctness checker, and feedback-generation components must operate consistently for the complete system to be useful.

These limitations should be considered during implementation and subsequent evaluation.

7.8 Future Testing and Improvement

Further work should include implementing the proposed modules, preparing a documented set of test programs, and executing the test cases defined in this chapter.

The resulting observations should be recorded in a test-results table containing the test case ID, input, expected output, actual output, status, and relevant remarks. Quantitative results should include the dataset size, evaluation configuration, and calculation procedure.

Future evaluation may also compare the proposed framework with a baseline approach using the same test examples and evaluation conditions. This would help determine whether the combination of review-necessity assessment, correctness checking, and educational feedback provides measurable benefits.

7.9 Conclusion

This chapter establishes the testing and evaluation approach for Code Mentor GPT. The proposed methodology covers unit testing, integration testing, functional testing, review-quality evaluation, correctness checking, and performance measurement.

BERTScore can support the semantic evaluation of generated comments, while predefined test cases and manual examination are necessary to assess technical correctness and educational usefulness. The actual effectiveness of the proposed system must be established through implementation, execution of the planned tests, and analysis of the resulting evidence.

CONCLUSION

The proposed project, CODE MENTOR GPT: A HYBRID AI FRAMEWORK FOR EDUCATIONAL CODE REVIEW AND INTELLIGENT COMMENT GENERATION, aims to improve the quality of programming education by providing intelligent, meaningful, and educational feedback on students' source code. The framework is designed to integrate Large Language Models (LLMs), particularly GPT-4o, with a structured code-review approach to identify programming errors, explain coding issues, and encourage better programming practices.

The proposed system incorporates a Review Necessity Chain (RNC) to determine whether detailed code review is required, helping reduce unnecessary processing and token consumption. The Code Correctness Check Module (CCM) is intended to validate program correctness using test cases before generating detailed feedback. In addition, the framework aims to provide educational comments on errors, code quality, optimization opportunities, and programming practices without directly supplying complete solutions. Intelligent comment generation for classes, functions, and variables is also considered an extension of the framework. The proposed evaluation approach includes test-case-based correctness checking and BERTScore-based comparison of generated feedback with reference reviews. These evaluation methods are intended to assess the framework's functionality and the semantic similarity of its generated comments. However, the effectiveness and performance of the proposed framework must be established through implementation and experimental evaluation.

In conclusion, Code Mentor GPT presents a research-oriented approach to combining AI-based code review with educational feedback generation. By emphasizing correctness checking, relevant feedback, and efficient use of language models, the framework has the potential to support students in understanding programming mistakes and improving their coding skills. Future work will focus on implementing the proposed modules, conducting systematic experiments, evaluating feedback quality, and integrating the framework into a practical educational environment.

Program Outcome Achieved: PO1, PO2, PO3, PO4, PO5, PO7, PO8, PO9, PO11.

FUTURE SCOPE

1. Enhanced Personalized Code Feedback: Future development can improve the system by generating feedback according to the learner's programming skill level and learning progress. The system can provide simpler explanations for beginners and more detailed suggestions for advanced learners, making the review process more suitable for individual learning needs.

2. Advanced Code Understanding: The proposed system can be further enhanced with advanced language models and improved code-understanding techniques. These approaches can help the system better understand complex program structures, coding logic, and relationships between different parts of a source code.

3. Adaptive Learning Support: Future versions can incorporate learner history and previous coding submissions to provide continuous and personalized guidance. The system could identify repeated mistakes and generate targeted feedback to help students gradually improve their programming skills.

4. Improved Code Correctness and Review: Future work can strengthen the correctness-checking and review process by supporting a wider range of programming problems and programming languages. More comprehensive test cases and code-analysis techniques can be incorporated to identify different types of errors while maintaining meaningful educational feedback.

5. Scalable Educational Deployment: The system can be extended for large-scale use in schools and online programming-learning platforms. Improvements in processing efficiency, review filtering, and feedback generation can help reduce instructor workload while supporting a larger number of student submissions and providing timely code-review assistance.

REFERENCES

[1] Ministry of Education, Singapore, “Transforming Education Through Technology,” 2024. [Online]. Available: https://www.moe.gov.sg/education-in-sg/educational-technology-journey/edtech-masterplan. Accessed: May 3, 2025.

[2] Ministry of Education, “The 2022 Revised Elementary School Curriculum,” 2024. [Online]. Available: https://www.gne.go.kr/upload_data. Accessed: Dec. 2023.

[3] Ministry of Education and Culture, Finland, *Policies for the Digitalisation of Education and Training Until 2027*, 2023. [Online]. Available: http://urn.fi/URN:ISBN:978-952-263-945-5. Accessed: May 2025.

[4] Ministry of Science and ICT, “Global Digital Competency Education,” 2021. [Online]. Available: https://www.nia.or.kr/cnt/edu. Accessed: Dec. 2023.

[5] OECD, *OECD Skills Outlook 2019: Thriving in a Digital World*. Paris, France: OECD Publishing, May 2019.

[6] C3Coding, “The Online Judge System for Elementary and Secondary School Students,” 2023. [Online]. Available: https://judge.c3coding.com/. Accessed: Dec. 2023.

[7] W. Dai et al., “Can large language models provide feedback to students?,” in *Proc. IEEE Int. Conf. Adv. Learn. Technol. (ICALT)*, Jul. 2023, pp. 323–325.

[8] B. McDanel and E. Novak, “Designing LLM-resistant programming assignments,” in *Proc. 56th ACM Tech. Symp. Comput. Sci. Educ.*, Feb. 2025, pp. 756–762, doi: 10.1145/3641554.3701872.

[9] S. Saylam, N. Duman, Y. Yildirim, and K. Satsevich, “Empowering education with AI: Addressing ethical concerns,” *London J. Soc. Sci.*, vol. 6, no. 6, pp. 39–48, Sep. 2023.

[10] Y. Xie, S. Wu, and S. Chakravarty, “AI meets AI: Artificial intelligence and academic integrity,” in *Proc. 24th Annu. Conf. Inf. Technol. Educ.*, Oct. 2023, pp. 79–83.

[11] R. Singh, S. Gulwani, and A. Solar-Lezama, “Automated feedback generation for introductory programming assignments,” in *Proc. 34th ACM SIGPLAN Conf. Program. Lang. Design Implement. (PLDI)*, Jun. 2013, pp. 15–26, doi: 10.1145/2491956.2462195.

[12] M.-F. Wong, S. Guo, C.-N. Hang, S.-W. Ho, and C.-W. Tan, “Natural language generation and understanding of big code for AI-assisted programming,” *Entropy*, vol.

25, no. 6, Art. no. 888, Jun. 2023, doi: 10.3390/e25060888.

[13] J. P. Amos et al., “A bibliometric review on leveraging LLMs for programming education,” *IEEE Access*, vol. 13, pp. 58364–58393, 2025.

[14] M. Kazemitabaar et al., “CodeAid: Evaluating a classroom deployment of an LLM-based programming assistant,” in *Proc. CHI Conf. Hum. Factors Comput. Syst.*, May 2024, pp. 1–20, doi: 10.1145/3613904.3642773.

[15] S. Sahai, U. Z. Ahmed, and B. Leong, “Improving the coverage of GPT for automated feedback,” in *Proc. NeurIPS Workshop Gener. AI Educ. (GAIED)*, vol. 46, 2023, p. 1. [Online]. Available: https://neurips.cc/virtual/2023/79073.

[16] H. Nguyen and V. Allan, “Using GPT-4 to provide tiered, formative code feedback,” in *Proc. 55th ACM Tech. Symp. Comput. Sci. Educ.*, Mar. 2024, pp. 958–964. [Online]. Available: https://dl.acm.org/doi/10.1145/3626252.3630960.

[17] A. S. Crandall, B. Fischer, and J. P. Crandall, “WIP: ARTful insights from a pilot study on GPT-based automatic code reviews,” in *Proc. IEEE Frontiers Educ. Conf. (FIE)*, Oct. 2024, pp. 1–5. [Online]. Available: https://ieeexplore.ieee.org/document/10893407.

[18] A. S. Crandall, G. Sprint, and B. Fischer, “GPT models as a code review feedback tool in computer science programs,” *J. Comput. Sci. Colleges*, vol. 39, no. 1, pp. 38–47, Oct. 2023. [Online]. Available: https://dl.acm.org/doi/10.5555/3636517.3636522.

[19] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, “BERTScore: Evaluating text generation with BERT,” arXiv:1904.09675, 2019.

[20] M. Cardona, R. Rodriguez, and K. Ishmael, *Artificial Intelligence and the Future of Teaching and Learning*. Washington, DC, USA: U.S. Department of Education, May 2025. [Online]. Available: https://www.ed.gov/sites/ed/files/documents/ai-report/ai-report.pdf. Accessed: May 2025.

[21] U.S. Department of Education, “AI Guidance and Use Case Inventory,” 2025. [Online]. Available: https://www.ed.gov/about/ed-overview/artificial-intelligence-ai-guidance. Accessed: May 3, 2025.

[22] British Informatics Olympiad, “Rules of the 2025 British Informatics Olympiad,” 2025. [Online]. Available: https://www.olympiad.org.uk/2025/rules.html. Accessed: May 2025.

[23] M. Jaber, “New innovations in higher education’s academic integrity,” in *Academic Integrity in the Age of Artificial Intelligence*. Hershey, PA, USA: IGI Global, Feb. 2024, pp. 281–300.

[24] College Board, “AP Computer Science A—The Exam,” AP Central, 2023. [Online]. Available: https://apcentral.collegeboard.org/courses/ap-computer-science-a/exam. Accessed: May 2025.

[25] S. Choi, D. Lee, J. Kim, Y. Jang, and H. Kim, “Designing LLM-based code reviewing learning environment,” *J. Korean Assoc. Comput. Educ.*, vol. 26, no. 5, pp. 1–11, Jul. 2023.

[26] J. Lu, L. Yu, X. Li, L. Yang, and C. Zuo, “LLaMA-Reviewer: Advancing code review automation with large language models,” arXiv:2308.11148, 2023.

[27] Q. Zhang et al., “A critical review of large language model on software engineering,” arXiv:2310.08879, 2023.

[28] H.-J. Moon, “Evaluation of user satisfaction and effectiveness of web (App)-based AI application services,” *J. Digit. Contents Soc.*, vol. 24, no. 10, pp. 2611–2617, Oct. 2023.

[29] T. Sun, J. He, X. Qiu, and X. Huang, “BERTScore is unfair: On social bias in language model-based metrics,” in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, vol. 1, Jan. 2022, pp. 3726–3739. [Online]. Available: https://aclanthology.org/2022.emnlp-main.245/.